
SICoxCURE

Single-Index/Cox Mixture Cure Model

R Package — Version 0.1.2

User Guide & Reference Manual

Developer: Mohamed Yassir Errahmani

GitHub: github.com/MohamedYassirErrahmani/SICoxCURE

Authors: Mohamed Yassir Errahmani & Maïlis Amico

Email: mohamed-yassir.errahmani@umontpellier.fr
mailis.amico@kuleuven.be

Reference: Amico, Van Keilegom & Legrand (2019)
Biometrics, **75**(2), 452–462
[doi:10.1111/biom.12999](https://doi.org/10.1111/biom.12999)

License: GPL (≥ 3)

Contents

1	Overview	2
1.1	The Model	2
1.2	Estimation	2
1.3	When to Use a Cure Model	2
2	Installation	3
2.1	From CRAN (Coming soon...)	3
2.2	From GitHub (development version)	3
2.3	Load the Package	3
3	Data Preparation	3
3.1	Required Data Structure	3
3.2	How to Organize Your Own Data	4
3.3	Using a data.frame	5
4	Fitting the Model: <code>sic_fit()</code>	5
4.1	Main Parameters	5
4.2	Workflow A: All-in-One (Recommended)	6
4.3	Workflow B: Step by Step	6
5	Output Object: <code>sic_fit</code>	7
5.1	<code>print()</code> method	7
5.2	<code>summary()</code> method	8
6	Bootstrap Standard Errors: <code>bootstrap_se()</code>	9
7	Graphical Functions	9
7.1	<code>plot_link()</code> : Estimated Link Function	9
7.2	<code>plot_survival()</code> : Latency Survival Curves	10
7.3	Integrated in <code>sic_fit()</code>	10
8	Predictions: <code>predict()</code>	10
8.1	Components of <code>pred</code>	11
9	Real Data Example: VERIDEX	12
10	Tips and Recommendations	13
10.1	Bandwidth Selection	13
10.2	Bootstrap	13
10.3	Parallel Computing	13
11	Function Reference (v0.1.2)	14
11.1	Public Functions	14
11.2	Internal Functions (accessible via <code>:::</code>)	14
12	Reference	15

1. Overview

The **SICoxCURE** package implements the **Single-Index/Cox (SIC) mixture cure model** proposed by Amico, Van Keilegom and Legrand (2019). It was developed and maintained by **Mohamed Yassir Errahmani** (University of Montpellier).

1.1 The Model

In survival analysis, a cure model is appropriate when a fraction of subjects will never experience the event of interest (the “cured” subjects). The SIC mixture cure model writes the population survival function as:

$$S(t \mid \mathbf{x}, \mathbf{z}) = \underbrace{1 - p(\mathbf{x})}_{\text{cured fraction}} + \underbrace{p(\mathbf{x}) \cdot S_u(t \mid \mathbf{z})}_{\text{uncured fraction}}$$

where:

- $p(\mathbf{x}) = g(\boldsymbol{\gamma}^\top \mathbf{x})$ is the **incidence** (probability of being uncured), modeled via a single-index with unknown link function g estimated nonparametrically using Nadaraya-Watson kernel smoothing.
- $S_u(t \mid \mathbf{z})$ is the **latency** (conditional survival of uncured subjects), modeled via a Cox proportional hazards model.
- The vectors $\mathbf{X}$ and $\mathbf{Z}$ can contain partially or completely different covariates.

1.2 Estimation

Estimation is performed by maximum likelihood using an **Expectation-Maximization (EM) algorithm**:

- **E-step**: computes posterior weights $W_i = P(B_i = 1 \mid \text{data}, \theta^{(m-1)})$
- **M-step (incidence)**: maximizes $\tilde{L}_1$ to update $\hat{\boldsymbol{\gamma}}$ and $\hat{g}(\cdot)$ (Nadaraya-Watson estimator)
- **M-step (latency)**: maximizes $\tilde{L}_2$ (partial likelihood) to update $\hat{\boldsymbol{\beta}}$ and $\hat{\Lambda}_0$

The bandwidth h for the kernel estimator is selected at each iteration by **likelihood cross-validation** over a user-specified grid.

1.3 When to Use a Cure Model

Conditions for a cure model

1. Event rate < 50% (many potential long-term survivors)
2. Visible plateau in the Kaplan-Meier estimator (> 10%)
3. Sufficient follow-up time beyond the last observed event

2. Installation

2.1 From CRAN (Coming soon...)

```
1 install.packages("SICoxCURE")
```

Listing 1: Installation from CRAN

2.2 From GitHub (development version)

```
1 # Install devtools if needed
2 install.packages("devtools")
3
4 # Install SICoxCURE from GitHub
5 devtools::install_github("MohamedYassirErrahmani/SICoxCURE")
```

Listing 2: Installation from GitHub

2.3 Load the Package

```
1 library(SICoxCURE)
2
3 # Check version
4 packageVersion("SICoxCURE")
```

Listing 3: Loading SICoxCURE

Listing 4: Expected output

```
[1] '0.1.2'
```

```
1 ls("package:SICoxCURE")
```

Listing 5: List available functions

```
[1] "bootstrap_se" "plot_link" "plot_survival" "sic_fit"
```

```
1 methods(class = "sic_fit")
```

Listing 6: Check S3 methods

```
[1] predict print summary
```

3. Data Preparation

3.1 Required Data Structure

The `sic_fit()` function requires the following inputs:

Argument	Type	Description
Y	Numeric vector (n)	Follow-up times $Y_i = \min(T_i, C_i)$
delta	Integer vector (n)	Event indicators (1=event, 0=censored)
X	Matrix ($n \times d$)	Incidence covariates for $\gamma^\top \mathbf{X}$
Z	Matrix ($n \times q$)	Latency covariates for Cox model

3.2 How to Organize Your Own Data

Before fitting the model, the user must prepare three objects:

- **Y**: a numeric vector of follow-up times $Y_i = \min(T_i, C_i)$, one value per subject.
- **delta**: an integer vector of event indicators: 1 if the event was observed, 0 if censored.
- **X**: a numeric matrix with n rows and d columns, where each column is one incidence covariate (used to model the uncure probability).
- **Z**: a numeric matrix with n rows and q columns, where each column is one latency covariate (used in the Cox model for uncured subjects).

Important

- **X** and **Z** can contain the same covariates, partially overlapping ones, or completely different ones.
- Covariates in **X** are automatically standardized (centered and scaled) by `sic_fit()` when `standardize_X = TRUE` (default). This follows the recommendation of Amico, Van Keilegom and Legrand (2019) [doi:10.1111/biom.12999](https://doi.org/10.1111/biom.12999).
- Missing values must be removed before calling `sic_fit()`.

Example with a real dataset:

```

1 # Load your data
2 dat <- read.csv("your_data.csv")
3
4 # Remove missing values
5 dat <- na.omit(dat)
6
7 # Prepare vectors and matrices
8 Y <- dat$follow_up_time # follow-up times
9 delta <- dat$event_indicator # 1=event, 0=censored
10
11 # Incidence covariates (X)
12 X <- as.matrix(dat[, c("age", "ER_status",
13                        "tumor_size")])
14
15 # Latency covariate (Z)
16 Z <- as.matrix(dat[, "ER_status", drop = FALSE])
17
18 # Check dimensions
19 cat(sprintf("n=%d | Events=%.1f%% | Censored=%.1f%%\n",

```

```

20     nrow(dat),
21     100 * mean(delta),
22     100 * mean(delta == 0)))

```

Listing 7: Preparing your own data

3.3 Using a data.frame

```

1  dat <- data.frame(Y=Y, delta=delta,
2                      X1=X[,1], X2=X[,2],
3                      X3=X[,3], X4=X[,4],
4                      Z=Z[,1])
5
6  fit <- sic_fit(
7    data      = dat,
8    col_Y     = "Y",
9    col_delta = "delta",
10   cols_X    = c("X1", "X2", "X3", "X4"),
11   cols_Z    = "Z"
12 )

```

Listing 8: data.frame interface

4. Fitting the Model: sic_fit()

4.1 Main Parameters

Parameter	Default	Description
Y	—	Follow-up times
delta	—	Event indicators
X	—	Incidence covariates
Z	—	Latency covariates
kernel	"epanechnikov"	Kernel function
h_range	c(0.4, 1.0)	Bandwidth search interval
h_step	0.001	Bandwidth grid step (601 values)
constraint	"gamma1"	Identifiability: fix $\gamma_1 = 1$
tol	1e-5	EM convergence tolerance
max_iter	500L	Maximum EM iterations
standardize_X	TRUE	Standardize X before fitting
bootstrap	FALSE	Compute SE automatically
B	250L	Bootstrap samples
boot_seed	NULL	Seed for bootstrap
plot	FALSE	Auto-plot after fitting
plot_Z	NULL	Z values for survival plot
plot_labels	c("Z=0", "Z=1")	Labels for survival plot

verbose 1L 0=silent, 1=progress, 2=detail

4.2 Workflow A: All-in-One (Recommended)

```

1 fit <- sic_fit(
2   Y           = Y,
3   delta       = delta,
4   X           = X,
5   Z           = Z,
6   h_step      = 0.05,      # coarser grid for speed
7   bootstrap   = TRUE,      # compute SE automatically
8   B           = 250L,      # 250 bootstrap samples (article)
9   boot_seed    = 42L,      # for reproducibility
10  plot        = TRUE,      # auto-plot after fitting
11  plot_Z       = matrix(c(0, 1), ncol = 1),
12  plot_labels  = c("Z = 0", "Z = 1"),
13  verbose      = 1L        # show progress bar
14 )

```

Listing 9: Fitting with integrated bootstrap and plots

Console output (observed during fitting):

```

=== Fitting SIC Mixture Cure Model ===
Step 1/3: Data validation and preparation...
X standardized (centered and scaled): 4 column(s). [Article]
  X1: mean=-0.010, sd=1.010 -> after: mean=0, sd=1
  ...
=== Data summary ===
  n=200 (no missing data)
  Events=89 (44.5%) | Censored=111
  tau=3.4521 | Plateau=48 (24.0%)
  X [X1,X2,X3,X4] | Z [Z]
=====
Step 2/3: Initialisation...
Step 3/3: EM algorithm...
--- Starting EM algorithm ---
Bandwidth grid: [0.40, 1.00] step=0.0500 (13 values)
|=====| 100%
[32m Checkmark Model converged successfully at iteration 8
      (criterion = 3.21e-06 < tol = 1e-05)[0m
  Final complete log-likelihood : -412.3847
Computing bootstrap standard errors...
Bootstrap standard errors: B=250, n=200, parallel=TRUE

```

Convergence message

When the model converges, a green message is displayed: **✓Model converged successfully at iteration N** along with the convergence criterion value.

4.3 Workflow B: Step by Step

```

1 # Step 1: Fit the model
2 fit <- sic_fit(Y, delta, X, Z,

```

```

3         h_step = 0.05,
4         verbose = 1L)
5
6 # Step 2: Bootstrap standard errors
7 se <- bootstrap_se(fit,
8                   B      = 250L,
9                   parallel = TRUE,
10                  seed    = 42L,
11                  verbose  = TRUE)
12
13 # Step 3: Summary with p-values
14 summary(fit, se = se)
15
16 # Step 4: Graphical outputs
17 plot_link(fit)
18 plot_survival(fit,
19              newZ      = matrix(c(0, 1), ncol = 1),
20              group_labels = c("Z = 0", "Z = 1"))
21
22 # Step 5: Predictions
23 pred <- predict(fit,
24                newX = X[1:5, ],
25                newZ = Z[1:5, , drop=FALSE],
26                type = "all")

```

Listing 10: Step-by-step workflow

5. Output Object: sic_fit

The `sic_fit()` function returns an S3 object of class "sic_fit" containing:

Element	Type	Description
gamma	Vector (d)	Estimated incidence coefficients $\hat{\gamma}$
beta	Vector (q)	Estimated latency coefficients $\hat{\beta}$
p_hat	Vector (n)	Estimated uncure probabilities $\hat{p}(\mathbf{x}_i)$
Su_hat	Vector (n)	Estimated latency $\hat{S}_u(Y_i \mathbf{z}_i)$
breslow	List	Breslow estimator: times, Λ_0 , S_0
h	Scalar	Selected bandwidth h_{CV}
W	Vector (n)	Final EM weights $\hat{W}_i$
loglik	Vector	Complete log-likelihood per iteration
converged	Logical	Did the EM algorithm converge?
n_iter	Integer	Number of EM iterations
se	List	Bootstrap SE (if <code>bootstrap=TRUE</code>)

5.1 print() method

```
1 print(fit)
```


Listing 11: Compact display

```

=== SIC Mixture Cure Model ===

Call: sic_fit(Y = Y, delta = delta, X = X, Z = Z, h_step = 0.05,
  verbose = 1L)

n = 200 | events = 89 | kernel = epanechnikov | h = 0.5500
Convergence: YES in 8 iterations (crit = 3.21e-06)

Incidence coefficients (gamma):
      X1      X2      X3      X4
1.0000 -0.4832  0.2876 -0.1954

Latency coefficients (beta):
      Z
1.3847

Use summary() for standard errors and p-values.

```

5.2 summary() method

```

1 summary(fit)      # uses fit$se if bootstrap=TRUE
2 # or
3 summary(fit, se = se)  # with external bootstrap SE

```

Listing 12: Summary with p-values

```

=== Summary: SIC Mixture Cure Model ===

Call: sic_fit(Y = Y, delta = delta, X = X, Z = Z, ...)

Data:
  n = 200 | Events: 89 (44.5%) | Cure threshold tau = 3.4521
  Plateau: 48 observations (24.0%)

Model:
  Kernel: epanechnikov | Bandwidth h = 0.5500
  Constraint: 'gamma1' | Standardized X: yes

Convergence: YES in 8 iterations (crit = 3.21e-06, tol = 1e-05)

Incidence (gamma) -- single-index coefficients:
  Estimate Std.Error z.value Pr(>|z|)
X1      1.0000      --      --      --
X2    -0.4832    0.1943   -2.487   0.0129  *
X3     0.2876    0.2341    1.229   0.2192
X4    -0.1954    0.1876   -1.042   0.2974

Latency (beta) -- Cox model coefficients:
  Estimate Std.Error z.value Pr(>|z|)
Z       1.3847    0.2134    6.489   <0.001  ***

Signif. codes: ' <0.001' '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1

```

p-values

P-values below 0.001 are displayed as <0.001 instead of 0. Standard errors require bootstrap (either via `bootstrap=TRUE` in `sic_fit()` or via a separate call to `bootstrap_se()`).

6. Bootstrap Standard Errors: `bootstrap_se()`

```

1 se <- bootstrap_se(
2   fit      = fit,
3   B        = 250L,          # as in Amico et al. (2019)
4   parallel = TRUE,          # use multiple CPU cores
5   ncores   = NULL,          # auto: all cores minus 1
6   seed     = 42L,           # reproducibility
7   verbose  = TRUE
8 )

```

Listing 13: Computing bootstrap SE (as in the article, B=250)

```

Bootstrap standard errors: B=250, n=200, parallel=TRUE
Using 7 cores
This may take several minutes...
|=====| 100%

Done. 250/250 iterations succeeded.

```

```

1 print(se)

```

Listing 14: Display bootstrap SE

```

=== Bootstrap Standard Errors (SICoxCURE) ===

Bootstrap samples : 250 / 250 succeeded

Incidence (gamma):
      X1      X2      X3      X4
0.0000 0.1943 0.2341 0.1876

Latency (beta):
      Z
0.2134

Use summary(fit, se = <this object>) for p-values.

```

7. Graphical Functions

7.1 `plot_link()`: Estimated Link Function

```

1 plot_link(fit,
2           main = "Estimated link function g",
3           col   = "blue",
4           lwd   = 2)

```

Listing 15: Plotting the estimated link function g

This function plots $\hat{g}(u)$ against the single index $\hat{\gamma}^\top \mathbf{x}_i$ for the training observations, sorted by index value. A horizontal dashed line indicates the mean uncure probability.

7.2 plot_survival(): Latency Survival Curves

```

1 plot_survival(
2   fit,
3   newZ      = matrix(c(0, 1), ncol = 1),
4   group_labels = c("Z = 0", "Z = 1"),
5   cols      = c("red", "blue"),
6   main      = "Estimated latency survival"
7 )

```

Listing 16: Plotting latency survival $S_u(t|z)$

This function plots $\hat{S}_u(t | \mathbf{z})$ for specified values of $\mathbf{Z}$. It uses the Breslow estimator stored in `fit$breslow` and the estimated $\hat{\beta}$.

7.3 Integrated in sic_fit()

Both graphical functions can be called automatically by setting `plot = TRUE` in `sic_fit()`:

```

1 fit <- sic_fit(Y, delta, X, Z,
2               plot      = TRUE,
3               plot_Z    = matrix(c(0, 1), ncol = 1),
4               plot_labels = c("Z = 0", "Z = 1"))

```

Listing 17: Automatic plots within `sic_fit()`

8. Predictions: predict()

```

1 # Generate new observations
2 set.seed(99)
3 X_new <- matrix(rnorm(10 * 4), 10, 4)
4 Z_new <- matrix(rbinom(10, 1L, 0.6), 10, 1)
5
6 # Incidence only
7 p_hat <- predict(fit,
8                 newX = X_new,
9                 newZ = Z_new,
10                 type = "incidence")
11

```

```

12 # All predictions
13 pred <- predict(fit,
14                 newX      = X_new,
15                 newZ      = Z_new,
16                 t_eval    = c(0.5, 1, 2, 5),
17                 type      = "all")

```

Listing 18: Predicting for new observations

8.1 Components of pred

Component	Type	Description
pred\$incidence	Vector (m)	$\hat{p}(\mathbf{x}_i)$ for each patient
pred\$latency	Matrix ($m \times T$)	$\hat{S}_u(t \mathbf{z}_i)$
pred\$survival	Matrix ($m \times T$)	$\hat{S}(t \mathbf{x}_i, \mathbf{z}_i)$
pred\$t_eval	Vector (T)	Time points
pred\$index	Vector (m)	$\hat{\gamma}^\top \mathbf{x}_i$

```

1 tab <- data.frame(
2   Patient  = 1:10,
3   Index    = round(pred$index, 4),
4   p_x      = round(pred$incidence, 4),
5   S_t1     = round(pred$survival[, 2], 4), # t=1
6   S_t5     = round(pred$survival[, 4], 4)  # t=5
7 )
8 colnames(tab) <- c("Patient", "Index", "p(x)",
9                   "S(t=1|x,z)", "S(t=5|x,z)")
10 print(tab)

```

Listing 19: Displaying predictions as a table

	Patient	Index	p(x)	S(t=1 x,z)	S(t=5 x,z)
1	1	1.2138	0.9960	0.004	0.004
2	2	2.0503	1.0000	0.021	0.021
3	3	-0.9207	0.8973	0.103	0.103
4	4	-5.4874	0.0000	1.000	1.000
5	5	-8.7415	0.8374	0.163	0.163
6	6	-2.1682	0.4968	0.503	0.503
7	7	-2.0149	0.5450	0.455	0.455
8	8	-4.8327	0.0000	1.000	1.000
9	9	-4.0791	0.1621	0.838	0.838
10	10	0.5943	0.9911	0.030	0.030

Interpreting predictions

- **Patient 4 / 8:** $p(x) \approx 0 \Rightarrow \text{cured} \Rightarrow S(t|x, z) = 1.000$
- **Patient 1 / 2:** $p(x) \approx 1 \Rightarrow \text{uncured} \Rightarrow \text{poor prognosis}$
- **Patient 6:** $p(x) \approx 0.50 \Rightarrow \text{uncertain (50/50)}$

The formula $S = 1 - p(x) + p(x) \cdot S_u$ guarantees that cured patients ($p \rightarrow 0$) always

have $S \rightarrow 1$.

9. Real Data Example: VERIDEX

This example reproduces the breast cancer application of Amico et al. (2019), Section 4. The dataset consists of 286 patients with lymph-node-negative breast cancer (Wang et al., 2005). The event of interest is distant metastasis.

```

1 library(SICoxCURE)
2
3 # Load data
4 dat <- read.table("VERIDEX-survival.txt",
5                   header = TRUE, sep = "\t")
6 colnames(dat) <- c("PID", "GEO", "lymph", "time",
7                   "status", "ER", "brain")
8 dat$ER_bin <- as.integer(dat$ER == "ER+")
9
10 cat(sprintf("n=%d | Events=%d (%.1f%%)\n",
11             nrow(dat), sum(dat$status),
12             100*mean(dat$status)))
13
14 # Train/test split (2/3 -- 1/3 as in the article)
15 set.seed(2019)
16 idx <- sample(nrow(dat), floor(2/3 * nrow(dat)))
17 train <- dat[idx, ]
18 X_tr <- as.matrix(train[, "ER_bin", drop=FALSE])
19 Z_tr <- as.matrix(train[, "ER_bin", drop=FALSE])
20
21 # Fit SICoxCURE (B=250 as in the article)
22 fit <- sic_fit(
23   Y           = train$time,
24   delta       = train$status,
25   X           = X_tr,
26   Z           = Z_tr,
27   h_step      = 0.001,           # article default
28   bootstrap   = TRUE,
29   B           = 250L,
30   boot_seed   = 2019L,
31   plot        = TRUE,
32   plot_Z      = matrix(c(0, 1), ncol = 1),
33   plot_labels = c("ER- (Z=0)", "ER+ (Z=1)"),
34   verbose     = 1L
35 )
36
37 # Summary (reproduces Table 3 of the article)
38 summary(fit)

```

Listing 20: VERIDEX application (as in the article)

```

n=286 | Events=107 (37.4%)

=== Summary: SIC Mixture Cure Model ===

Incidence (gamma):
      Estimate Std.Error z.value Pr(>|z|)
ER_bin      1.0000      --      --      --

Latency (beta):
      Estimate Std.Error z.value Pr(>|z|)
ER_bin     -0.9279     0.2753  -3.370  <0.001 ***

Signif. codes:  ' <0.001'  '***'  0.001  '**'  0.01  '*'  0.05  '.'  0.1

```

10. Tips and Recommendations

10.1 Bandwidth Selection

If $h_{CV} = h_{\min}$ or $h_{CV} = h_{\max}$, the true optimum may lie outside the search interval. Extend `h_range` accordingly.

```

1 # Quick exploration (13 values)
2 fit_fast <- sic_fit(Y, delta, X, Z,
3                   h_step = 0.05)
4
5 # Final publication result (601 values, as in article)
6 fit_final <- sic_fit(Y, delta, X, Z,
7                   h_step = 0.001)

```

Listing 21: Bandwidth tuning

10.2 Bootstrap

```

1 # Quick exploration
2 se_quick <- bootstrap_se(fit, B = 30L,
3                         parallel = FALSE)
4
5 # Publication quality (as in article)
6 se_final <- bootstrap_se(fit, B = 250L,
7                         parallel = TRUE,
8                         seed = 42L)

```

Listing 22: Bootstrap recommendations

10.3 Parallel Computing

```
1 parallel::detectCores() # total cores on your PC
```

Listing 23: Check available CPU cores

Parallel computing

`parallel = TRUE` uses all available CPU cores minus one, keeping one core free for the operating system. On a machine with 8 cores, this reduces bootstrap time from ~ 8 hours ($B = 250$ sequential) to ~ 1 hour.

11. Function Reference (v0.1.2)

11.1 Public Functions

Table 2: Public functions in SICoxCURE v0.1.2

Function	File	Description
<code>sic_fit()</code>	<code>sic_fit.R</code>	Main function. Fits the SIC model via EM.
<code>bootstrap_se()</code>	<code>bootstrap.R</code>	Bootstrap SE for $\hat{\gamma}$ and $\hat{\beta}$.
<code>plot_link()</code>	<code>plots.R</code>	Plot the estimated link function $\hat{g}(u)$.
<code>plot_survival()</code>	<code>plots.R</code>	Plot latency survival $\hat{S}_u(t \mathbf{z})$.
<code>predict.sic_fit()</code>	<code>predict.R</code>	Predict $\hat{p}(\mathbf{x})$, $\hat{S}_u$, $\hat{S}$ for new data.
<code>print.sic_fit()</code>	<code>sic_fit.R</code>	Compact display of a <code>sic_fit</code> object.
<code>summary.sic_fit()</code>	<code>sic_fit.R</code>	Coefficient table with SE and p-values.
<code>print.sic_bootstrap()</code>	<code>bootstrap.R</code>	Display a <code>sic_bootstrap</code> object.

11.2 Internal Functions (accessible via `:::`)

Table 3: Key internal functions in SICoxCURE v0.1.2

File / Function	Role
<code>em_algorithm.R</code>	
<code>run_em()</code>	Main EM loop: $E \rightarrow M1 \rightarrow M2 \rightarrow M3 \rightarrow$ convergence check
<code>estep.R</code>	
<code>estep()</code>	E-step: computes $W_i = P(B_i = 1 \text{data}, \theta)$
<code>mstep_incidence.R</code>	
<code>mstep_incidence()</code>	M-step for γ : L-BFGS-B with Nelder-Mead fallback
<code>neg_loglik_incidence()</code>	Objective function $-\log(\tilde{L}_1)$ for <code>optim()</code>
<code>mstep_latency.R</code>	
<code>mstep_latency()</code>	M-step for β : partial likelihood
<code>neg_partial_loglik()</code>	Objective function $-\log(\tilde{L}_2)$ for <code>optim()</code>
<code>nadaraya_watson.R</code>	

<code>nw_loo()</code>	Leave-one-out Nadaraya-Watson estimator of g
<code>nw_predict()</code>	Nadaraya-Watson prediction for new observations
<code>select_bandwidth()</code>	Cross-validation bandwidth selection
utils.R (17 functions)	
<code>kernel_epanechnikov()</code>	Epanechnikov kernel $K(u) = 3(1 - u^2/5)/(4\sqrt{5}) \cdot \mathbf{1}(u^2 \leq 5)$
<code>breslow_estimator()</code>	Weighted Breslow estimator for $\hat{\Lambda}_0$
<code>eval_Su()</code>	Evaluate $S_u(t \mathbf{z})$ on a time grid
<code>compute_index()</code>	Single index $u_i = \boldsymbol{\gamma}^\top \mathbf{x}_i$
...	...

12. Reference

Amico M, Van Keilegom I, Legrand C (2019). The single-index/Cox mixture cure model. *Biometrics*, **75**(2), 452–462. [doi:10.1111/biom.12999](https://doi.org/10.1111/biom.12999)

Wang Y, Klijn JGM, Zhang Y, et al. (2005). Gene-expression profiles to predict distant metastasis of lymph-node-negative primary breast cancer. *The Lancet*, **365**, 671–679.

Sy JP, Taylor JMG (2000). Estimation in a Cox proportional hazards cure model. *Biometrics*, **56**, 227–236.